

SPRINT TWO DELIVERABLES

Design Rationales:

Part 1: GameMap

In the part of GameMap, we will build a game class to create the game and it will check the game status to see the progress. The relationship between Game class and Board class is Aggregation. This design choice has better expansion possibilities, we can create different game maps in the future to have more games. Then we build the Board class which can get the information it needs from the Token class and the Position class. The responsibility of the Board class is to create the game map and display the tokens in the game map. The Position class will have 24 nodes. The position of the node will be shown as (x,y). The Token class' responsibility is to save the number of tokens for both players as static. Next, the Hint class will get the information from the Token class and Position class. Hint class will get the game status and legal next move position and print it in TutorialDisplay. In the Move class, the responsibility is to check the number of tokens and go to the correct MoveType. The last class is the Player class which will connect with the Game class, Token class, and Move class. The Player class has two players which are represented by two symbols (x,o). The Player class can be the owner of the token. Players can use the move class to move their tokens. The player class will get the input from the Game class and do the moving action. We have tried to use the factory method to update the game's turns. I think we can try to use the factory method to update the game's turns. But in this game the factory method will make code complex. It is not suitable for this game design.

Game: The project requires a Game entity, to create two Players and execute moves based on the user's input. This game must also have one Board, whose information it can pass to the Display.

Board: A board must exist to keep track of every new Move, all of the Tokens and their Positions. This information is crucial for the Game to receive and Display.

Player: Players are owners of Tokens *and use* their Tokens to make Moves.

Token: A Token is required to belong to one of two Players and a single Position on the board (if it has been placed already).

Move: Move is needed to facilitate the transfer of one Token from its current Position to another valid Position on the board. The player can directly make a Move of Type related to their current game stage (explained in more detail below).

Position: A Position must exist to identify as a single place on the board, which borders other Positions on the board.

Hint: The project requires a Hint entity so that it can find all valid potential Moves for a player's Token.

Part 2: GameBehavior

In the GameBehavior section, it has three subclasses, namely PutToken, Slide and Jump. These are all the movement actions of this game. We will use the single responsibility principle. We will write a check method in the GameBehavior. The relationship between MoveType class and PutToken, Slide and Jump classes is inherited. By this way, we can put the same method in Movetype class. At the same time, the Movetype class will have a check token method to decide move type. When the number of tokens changes, the player will choose a different move action. When the remaining pieces outside the board are greater than 0, the player must choose PutToken. when the remaining pieces on the board are greater than 3, the player must choose the slide action. When the remaining pieces on the board equal 3, the player must select the Jump action. When there are less than 3 pieces left on the board, the player loses the game. In the PutToken class, we will write a method to check for empty. In the Sliding class, we will write a method to check if NeighborLocation is empty. And the Jump class can be extended from the PutToken class, because they both have the same check method. After each move, there will be a CheckMill method, which will check if there is a mill. If it is true, it will return the MillAction. And players need to input a location to remove another player's token.

MoveType: MoveType organizes the three unique move styles belonging to the respective game stage the player is currently in. The Player can use the appropriate MoveType for their game stage.

Put, Slide, Jump: Each of these entities provide the tools to validate a player's move of *that* respective type.

Part 3: Display

In the part of Display design, we decided to divide the information of display into three divisions, which are MapDisplay, TutorialDisplay, and MessageDisplay. These three classes will inherit from Display class. Display class is an abstract class which includes a print method. This part uses the Single Responsibility Principle. In MapDisplay, we will print the board and show what's going on for the board. In the TutorialDisplay, we will print the legal next moving as a message type which will show the location and depends on the game state.(It will show all the legal moves) Every game state (like move, remove, jump) can show a legal next move as a location type when the player wants to get the hint. The last part is MessageDisplay, we will print the information about the game. Such as last moving, game

state, and turns. Display class as a fathering class of MapDisplay, TutorialDisplay, and MessageDisplay. It will have a print method to show the information.

Display: Display is needed to take board state information from the Game, so that the user can see each of the three types of display.

TutorialDisplay: TutorialDisplay uses unique visual elements to teach the Player the rules of the game, and how to navigate the UI.

MessageDisplay: MessageDisplay prints the latest change to the board state as a message to the screen.

MapDisplay: BoardDisplay displays the current state of the board to the screen.

Why?

1. Classes:

- 1.1. Action Class: The main reason for having an Action class is to create a general implementation of game mechanics. In the game, there are three different types of moves; however, a special type of game mechanics (like creating a mill) requires extra attention since it will induce the action of taking one of the pieces of the opponent out of the board.
- 1.2. Hint Class: Hint class is one of the advanced requirements and it is directly related to the Move Class. On the other hand, it requires the cooperation of multiple classes and in turn deserves to be treated as a class.

2. Relationships:

- 2.1. Game-Board (Composition): Since a board can be used for many different games, it has a meaning out of the Nine Men's Morris game context. This is why this relationship is a composition.
- 2.2. Board-Position(Aggregation): The positions have contextual meaning, they need the game board to exist. This is why this relationship is an aggregation.

3. Inheritance:

Since Inheritance establishes a firm bond between the parent class and child class and provides less flexibility, it is only used for definite relationships (like Action - MoveAction and RemoveAction).

4. Cardinalities:

- 4.1. Board-Token Cardinality (0...18): At the beginning of the game, the board is empty. The number of tokens can reach 18 on the board at any given time (9 tokens each). When the game is near the ending, the number of tokens can be at minimum 5 (3 for the winner, 2 for the loser).
- 4.2. Hint-Move Cardinality (0...54) : There are a lot of possibilities for the number of the hints. 54 hints are the maximum for having both players at the "Flying" stage, meaning both have 3 tokens, and there are 18 empty spaces on the board, giving $3 \times 18 = 54$. 0 is the minimum possible number of hints, for example the moment the game ends.

5. Design Pattern:

Since the project is just being implemented, it is better to focus on behavioural design patterns. One design pattern we are going to use to determine the turn of the players is the Observer design pattern, since it is heavily dependent on the activity of the players. We didn't discard any design patterns yet.

